

Sara Yuen, Amanda Dong

6)

Cannot change  $h$  without also changing  $dx$ , since  $dx = 0.8h$ .

Decreasing the mass of the particles, for example to 0.06, causes them to all drop down in a straight line. Increasing the mass of the particles, for example to 5.00, causes them to “burst” and fly quickly in all directions from the corner.

Smaller  $\rho_0$  (100.0) causes particles to burst and scatter.

Larger  $\rho_0$  (10000.0) initializes the pressure response much weaker, so particles move slowly.

Small  $k$  (300.0) makes particles drop down slightly to bottom of frame then stop.

Large  $k$  (30000.0) makes columns of particles expand away from each other slightly then stop.

Larger  $\mu$  (1.1) causes columns of particles to stick together like a chain.

Smaller  $\mu$  (0.01) causes particles to drop down slightly and stop.

Smaller gravity vector  $g$  ([0.0, -1.8]) makes columns of particles expand away from each other slightly then stop.

Larger gravity vector  $g$  ([0.0, -100.8]) causes simulation to run much faster.

Positive gravity vector  $g$  ([0.0, 9.8]) causes particles to float up slightly and stop.

Small  $dt$  ( $2.0e-10$ ) particles do not move.

Large  $dt$  ( $2.0e-1$ ) only a few particles visible moving rapidly mostly at the bottom of frame.

Small STEPS (1000) particles move very slight amount downward and stop.

Large STEPS (500000) similar to original.

SAVE\_EVERY (100) makes the output animation have fewer saved frames.

SAVE\_EVERY (1) makes the program slower because it writes many more CSV files.

Large  $dx$  (0.4) only 1 particle present because the spacing is too large for the initial region.

Small  $dx$  (0.004) creates a very large number of particles, making the simulation much slower.

Overall, the simulation was most sensitive to mass,  $\rho_0$ ,  $k$ , gravity, and  $dt$ . Extreme values made the particles unstable, while very small  $dt$  or weak gravity made the motion barely visible. SAVE\_EVERY mainly changed the output frequency rather than the physics.

7)

When using cubic spline, the particles “burst” and some form small clumps. This shows that changing the smoothing kernel changes the density estimate and pressure response, so the particle motion becomes less stable than the original kernel.

8)

450 particles. The particles still scatter a bit but overall they behave more like the original simulation. The original setup had 216 particles, so enlarging the setup more than doubled the particle count. This made the simulation slower because each step had more neighbor searches and force calculations.

9)

The time complexity for spatial hashing is approximately  $O(n)$  because the time complexity for

searching a hash map is  $O(1)$ , and each particle only checks nearby grid cells. Naive neighbor search is  $O(n^2)$  because every particle is compared with every other particle. Therefore, naive search takes more time, which would be more noticeable for large numbers of particles. In our tests, naive search produced similar neighbor results but took longer than spatial hashing.

10)

Euler-Cromer updates the velocity values first and then updates position using the new velocity, while Euler updates position using the previous velocity values. The Euler method can cause the magnitude of the eigenvalues to be greater than 1. Thus the algorithm is less stable because the oscillations are not bounded. Euler-Cromer is more stable because the oscillations stay more bounded

